

Backend — Spring Boot Application

▼ Relevant source files

Backend/STUDER/src/main/java/facu/studer/DTOs/user/UserResponseDTO.java

Backend/STUDER/src/main/java/facu/studer/StuderApplication.java

Backend/STUDER/src/main/java/facu/studer/config/AppConfig.java

Backend/STUDER/src/main/java/facu/studer/config/ApplicationStart.java

Backend/STUDER/src/main/java/facu/studer/config/I18nConfig.java

Backend/STUDER/src/main/java/facu/studer/config/RequestLoggingFilter.java

Backend/STUDER/src/main/java/facu/studer/config/WebClientConfig.java

The STUDER backend is a Java-based web application built on the **Spring Boot 3** framework. It provides a secure, multi-tenant REST API for user management, social interactions, threaded discussions, and real-time notifications. The application follows a strictly layered architecture to ensure maintainability and clear separation of concerns.

System Architecture

The backend operates as the central orchestrator of the STUDER ecosystem. It manages the business logic, interacts with a PostgreSQL database for persistence, and communicates with the Python-based **STUDER-MEDIA** service for image processing.

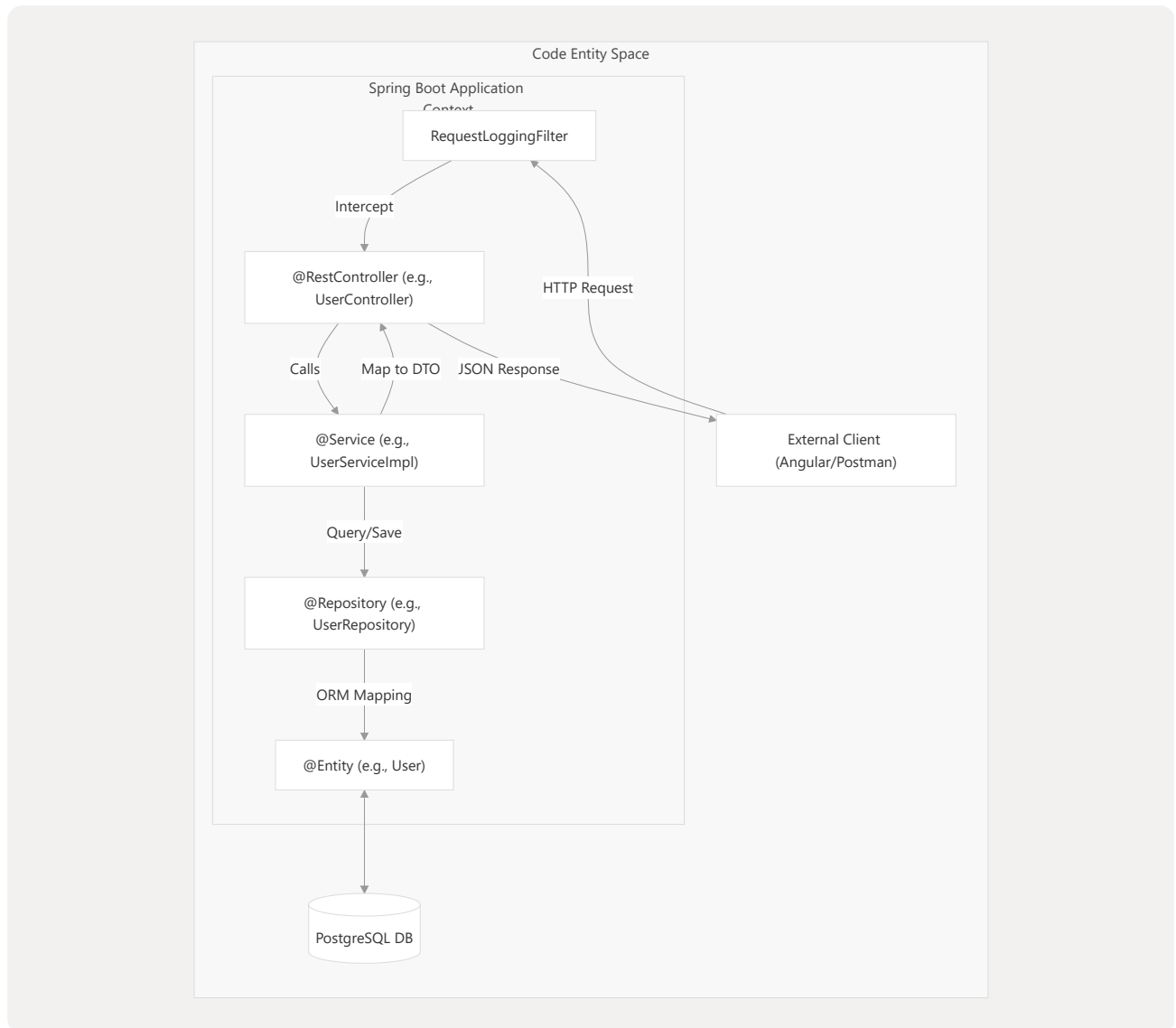
Layered Structure

The codebase is organized into four primary logical layers:

1. **Controller Layer:** Handles incoming HTTP requests, validates input, and delegates to services.
2. **Service Layer:** Contains the core business logic and transactional boundaries.
3. **Repository Layer:** Abstracts database access using Spring Data JPA.
4. **Entity Layer:** Represents the persistent data model (JPA Entities).

Data Flow Diagram

This diagram bridges the conceptual "Request-Response" cycle to the specific Java packages and classes used in the STUDER backend.



Sources:

- Backend/STUDER/src/main/java/facu/studer/StuderApplication.java 6-15
- Backend/STUDER/src/main/java/facu/studer/config/RequestLoggingFilter.java 16-49

Core Components & Configuration

The application initializes with several cross-cutting concerns that manage logging, internationalization, and startup behavior.

Startup & Lifecycle

Upon startup, the `StuderApplication` class launches the Spring context

`facu/studer/StuderApplication.java` 13-15 A custom listener, `ApplicationStart`, triggers an

ASCII banner and developer attribution once the context is refreshed

`facu/studer/config/ApplicationStart.java` 52-152

Request Observation

Every incoming request is intercepted by the `RequestLoggingFilter`. This component assigns a unique `requestId` to each transaction and logs the HTTP method, URI, status code (with representative emojis), and processing duration

`facu/studer/config/RequestLoggingFilter.java` 25-47

Internationalization (i18n)

The backend supports multi-language error messages and notifications. The `I18nConfig` class configures an `AcceptHeaderLocaleResolver`, which defaults to English but can resolve Spanish based on the `Accept-Language` header

`facu/studer/config/I18nConfig.java` 14-18

Sources:

- `Backend/STUDER/src/main/java/facu/studer/config/ApplicationStart.java` 17-52
- `Backend/STUDER/src/main/java/facu/studer/config/I18nConfig.java` 10-19
- `Backend/STUDER/src/main/java/facu/studer/config/RequestLoggingFilter.java` 16-50

Domain Subsystems

The backend logic is partitioned into several specialized modules. For detailed implementation specifics, refer to the child pages linked below.

Security & Authentication

Handles identity verification using JWT (JSON Web Tokens) and secure HttpOnly cookies for refresh tokens. It includes rate-limiting logic to prevent brute-force attacks.

- **Key Classes:** `JwtUtil`, `SecurityConfig`, `RefreshTokenService`.
- For details, see [Security & Authentication](#).

User Management

Manages user profiles, account settings, and the soft-delete mechanism. It also handles the integration with the media service for profile picture variants.

- **Key Classes:** `UserController`, `UserServiceImpl`, `UserResponseDTO`.
- For details, see [User Management API](#).

Social Graph

Implements the "follow" and "friend" model. A friendship is established only when two users follow each other.

- **Key Classes:** `Friend`, `FriendController`, `FriendServiceImpl`.
- For details, see [Social Graph — Friends & Followers](#).

Discussions & Messaging

The "Discussions" module provides threaded, public forums with tagging support, while the "Messaging" module provides private, friend-gated direct communication.

- **Key Classes:** `DiscussionController`, `DirectMessageService`, `TagService`.
- For details, see [Discussions API](#) and [Messaging API](#).

Notification System

A centralized system that generates alerts for social actions (follows), system updates, and new messages.

- **Key Classes:** `NotificationFactory`, `NotificationServiceImpl`, `LinkedType`.
- For details, see [Notification System](#).

Data Transfer Objects (DTOs)

The backend uses DTOs to decouple the internal database schema from the external API contract. DTOs frequently utilize the `SnakeCaseStrategy` to match the frontend's naming conventions.

DTO Example	Purpose	Key Fields
UserResponseDTO	Returns user profile data	id , username , profile_picture_webp_url
ApiError	Standardized error response	timestamp , message , status

Sources:

- Backend/STUDER/src/main/java/facu/studer/DTOs/user/UserResponseDTO.java 14-49